

# Sijil — Implementation Plan

**Version:** 1.0 **Owner:** Solo founder **Companion documents:** PRD.md ,  
SOFTWARE\_ARCHITECTURE\_AND\_TECHNICAL\_DESIGN.md , API\_SPEC.md

---

## 1. How to Use This Document

This plan sequences the “full scope” MVP (classification + consistency check + permit triage + CEPA origin) decided in the PRD, for a solo technical founder. The sequencing exists specifically to de-risk building four non-trivial capabilities alone: each phase produces something demoable and testable before the next begins, and the hardest technical risk (classification quality) is validated early rather than left until the end.

Two work streams run in parallel throughout: **engineering** (this document’s main focus) and **compliance-data research** (permit rules, CEPA rules) — the second doesn’t require a computer to make progress and should start immediately, not wait for Phase 5.

---

## 2. Phase Overview

Phase 0 → Environment & tooling setup  
Phase 1 → Foundation (auth, company/user model, document upload + storage)  
Phase 2 → Document extraction pipeline (OCR + LLM field extraction)  
Phase 3 → HS classification (core differentiator – validate this first)  
Phase 4 → Consistency checking  
Phase 5 → Permit triage  
Phase 6 → CEPA origin qualification  
Phase 7 → Reports, billing, polish  
Phase 8 → Pilot onboarding & feedback loop

Parallel, ongoing from day one:

Track A → Permit-rules data collection (MOCCAE, MOHAP, TDRA, ESMA/MoIAT, Ministry  
Track B → CEPA rules-of-origin data collection (curated agreement subset)

---

### 3. Phase 0 — Environment & Tooling Setup

**Objective:** A working local dev environment and empty-but-deployable skeleton, before writing any feature code.

**Tasks:**

- Initialize the repository per the structure in the architecture doc, Section 7
- Set up `docker-compose.yml` for local Postgres (with `pgvector` extension enabled) + Redis
- Set up the FastAPI app skeleton with the settings/config module (architecture doc Section 23) that fails fast on missing required environment variables
- Set up the Next.js frontend skeleton
- Set up Alembic for migrations
- Set up the CI pipeline skeleton (lint, type-check, test steps — architecture doc Section 25) even before there's much to test, so the discipline exists from commit one
- Set up mock implementations of `LLMClient` and `OCRClient` (architecture doc Section 13) so later development doesn't require live API credentials for every local run

**Definition of done:** `docker-compose up` + backend + frontend all run locally; an empty CI pipeline passes on a trivial commit; a health-check endpoint responds.

**Dependencies:** None — this is the starting point.

---

### 4. Phase 1 — Foundation

**Objective:** Authenticated multi-tenant accounts, with document upload and storage working end to end (no AI processing yet).

**Components built:**

- `auth` module: register, login, refresh, logout (API spec Section 4)
- `Company`, `User` models and repositories, with roles ( `company_admin`, `company_user`, `broker_admin`, `broker_user` )
- Tenant-isolation enforcement at the repository layer (architecture doc Section 14) — build this rule in from the start, since retrofitting it later across a growing codebase is far riskier

- `Shipment` creation and listing (API spec Section 6), with `status` starting at `created`
- Document upload endpoint (API spec Section 7) storing raw files to object storage, with file-type/size validation, but **no OCR/extraction yet** — status simply moves to `queued`

**Testing requirement:** Integration tests confirming a user from Company A can never read Company B's shipments or documents, even via a crafted request — this is the single most important test in this phase given the tenant-isolation architecture.

**Definition of done:** A user can register a company, log in, create a shipment, and upload a document that's durably stored — verified via a real (not mocked) object storage integration test.

**Dependencies:** Phase 0.

---

## 5. Phase 2 — Document Extraction Pipeline

**Objective:** Uploaded documents are OCR'd (if needed) and converted into structured, confidence-scored fields.

**Components built:**

- `document_processing_tasks.py` worker: OCR → LLM field extraction (architecture doc Section 11.1)
- `ExtractedDocument` schema and persistence
- Confidence-threshold logic: fields below threshold flagged `needs_manual_review`, never silently accepted
- `PATCH /shipments/{id}/documents/{id}` manual-correction endpoint (API spec Section 7)
- SSE status-streaming endpoint (API spec Section 14) so the frontend can show live processing status without aggressive polling

**Testing requirement:** A fixture set of real-format sample invoices/packing-lists/BOLs (anonymized or synthetic) covering: clean native-PDF invoices, scanned/low-quality images, multi-page documents, and at least one non-English (Arabic) sample — extraction accuracy tracked against this set from this phase onward.

**Definition of done:** 10 varied sample documents processed end-to-end with extracted fields matching manual verification; a deliberately corrupted file is rejected with a clear 422 before ever reaching the queue (architecture doc Section 31 edge case).

**Dependencies:** Phase 1 (needs shipments and stored documents to process).

---

## 6. Phase 3 — HS Classification (Core Differentiator)

**Objective:** Working HS code classification with visible confidence and reasoning — the product’s central value proposition, validated before anything else is layered on top.

**Components built:**

- One-time ETL: ingest the published UAE/GCC tariff schedule into the `TariffKBRepository` vector store (this is a distinct, somewhat tedious data-preparation task — budget real time for it, since classification quality is entirely bounded by this knowledge base’s completeness)
- `ClassificationService` (architecture doc Section 6.1): vector search → LLM reasoning with GRI rules → ranked candidates
- `POST /line-items/{id}/override` and `/reclassify` endpoints (API spec Section 8)
- **The golden-set regression suite** (architecture doc Section 21) — build this in this phase, not later; it’s the single most important test suite in the whole system and needs to exist before the classification prompt/logic gets iterated on repeatedly

**Testing requirement:** Golden-set of 200+ verified product-description-to-HS-code pairs; CI blocks any merge that drops accuracy below the last known-good baseline (architecture doc Section 25).

**Definition of done:** 10 real (sample) invoices’ line items classified end-to-end, with accuracy manually validated against correct codes; every classification either shows a clear confidence + reasoning, or is honestly flagged `requires_manual_review` — never a confident-looking wrong answer.

**Dependencies:** Phase 2 (needs extracted product descriptions as input). This phase should get disproportionate iteration time relative to its line count in this plan — it’s the product’s core bet.

---

## 7. Phase 4 — Consistency Checking

**Objective:** Cross-document discrepancy detection, relatively fast to build once the extraction pipeline exists.

**Components built:**

- `ConsistencyService` (architecture doc Section 6.2): field-by-field comparison with fuzzy matching for names/addresses
- `GET /shipments/{id}/discrepancies` and `/acknowledge` endpoints (API spec Section 9)
- Severity classification ( `blocking` / `warning` / `informational` ) and the rule that a shipment can't reach `analysis_complete` with an unresolved blocking discrepancy

**Testing requirement:** Unit tests per comparison rule, plus a suite of known-good and known-bad document sets (architecture doc Section 6.2).

**Definition of done:** A shipment with a deliberately introduced invoice/packing-list quantity mismatch is correctly flagged as `blocking` ; a shipment with only minor name-formatting differences (e.g., "LLC" vs "L.L.C.") is correctly treated as non-blocking.

**Dependencies:** Phase 2.

---

## 8. Phase 5 — Permit Triage

**Objective:** HS-code-to-regulator mapping, surfaced clearly to the user.

**Components built:**

- `PermitRulesRepository` reading from the versioned data files under `data/permit_rules/` (architecture doc Section 23) — **this depends on Track A (parallel research work) having produced usable data by this point**
- `PermitTriageService` (architecture doc Section 6.3)
- `GET /shipments/{id}/permits` endpoint (API spec Section 10), including the explicit `no_permits_required: true` state

**Testing requirement:** Table-driven tests against the known mappings already established in earlier research (food → MOCCA, telecom → TDRA, medical devices → ESMA/MoIAT + MOHAP, dual-use goods → Ministry of Economy, etc.)

**Definition of done:** A shipment containing a telecom device correctly surfaces the TDRA requirement; a shipment of ordinary non-regulated goods correctly returns `no_permits_required: true` rather than an ambiguous empty response.

**Dependencies:** Phase 3 (needs HS codes) + Track A having a usable initial dataset (see Section 12 below — this is the phase most likely to be blocked by data readiness rather than code).

---

## 9. Phase 6 — CEPA Origin Qualification

**Objective:** Preferential-tariff determination for the curated agreement subset, sequenced last among the four intelligence modules since its rules logic is the most complex.

**Components built:**

- `CEPAOriginService` (architecture doc Section 6.4): per-agreement rules-of-origin logic (regional value content vs. change-in-tariff-classification tests)
- `GET /line-items/{id}/origin` and `/origin/value-breakdown` endpoints (API spec Section 11), including the `"insufficient_data"` (never a false negative) handling
- Depends on Track B having produced usable rules data for at least the first 2–3 priority agreements

**Testing requirement:** Rule-specific unit tests per covered agreement; a specific test confirming that missing value-content data returns `insufficient_data` rather than silently defaulting to `qualifies: false`.

**Definition of done:** A sample shipment with known origin data correctly determines CEPA eligibility and estimated savings; a shipment with missing value data correctly prompts for the missing input rather than guessing.

**Dependencies:** Phase 3 (needs HS codes) + Track B data readiness.

---

## 10. Phase 7 — Reports, Billing, Polish

**Objective:** Package everything into a sellable product.

**Components built:**

- `ReportService` : PDF generation bundling classification, discrepancies, permits, and origin results (API spec Section 12)
- Billing integration: subscription checkout, webhook handling (API spec Section 13) — card details never touch Sijil's own servers
- Frontend polish: the results dashboard, manual-review/correction flows, the broker multi-company view
- Rate limiting tuning (API spec Section 15) — initial values, to be revisited against real usage

**Definition of done:** A user can go from signup → upload → full analysis → downloadable PDF report → active paid subscription, entirely self-serve, with no manual founder intervention required.

**Dependencies:** Phases 1–6 substantially complete.

---

## 11. Phase 8 — Pilot Onboarding & Feedback Loop

**Objective:** Get the first real (or near-real) shipments through the system with actual pilot customers, and use that to correct course before broader launch.

**Tasks:**

- Recruit 3–5 pilot customers (trading companies and/or brokers) willing to run real or realistic shipments through the tool alongside their existing process
- Compare Sijil’s output against what actually happened at customs (Yellow/Red channel outcome, actual permit requirements encountered, whether CEPA was correctly identified) — this is the first real validation of the golden-set and rules-data accuracy against ground truth, not just curated test data
- Feed corrections back into the golden-set, permit-rules data, and CEPA-rules data
- Revisit the numeric assumptions flagged as open in the PRD (Section 8 success metrics, Section 12 open questions) with real data now available

**Definition of done:** At least one pilot customer’s real shipment has gone through the full flow, with output compared against actual customs outcome, and at least one concrete correction has been made to the system as a result.

**Dependencies:** Phase 7.

---

## 12. Parallel Track A — Permit-Rules Data Collection

**What it is:** Building and maintaining the HS-code-to-regulator mapping table that Phase 5 depends on. This is research and structured data entry, not software engineering — it can start on day one, independent of any other phase.

**Suggested approach:**

1. Start from the category list already established in prior research (MOCCAE for food/plants/animals, MOHAP for pharma, TDRA for telecom, ESMA/MoIAT for regulated consumer goods and conformity marks, Ministry of Economy for dual-use/strategic

goods, Ministry of Interior for vehicles)

2. For each category, map the relevant HS code ranges/prefixes to the specific permit type and issuing authority
3. Store as versioned YAML/JSON files (architecture doc Section 23) with a clear changelog — since this data directly drives compliance advice given to users, every change should be reviewable, not an opaque edit
4. **This is not a one-time task.** Establish a recurring review cadence (e.g., quarterly) from the start — treat the first pass as a v1, not a finished dataset, per the explicit risk noted in the architecture doc (Sections 20 ADR-003, 31, 36, 37).

**Target for Phase 5 readiness:** Full coverage of the ~10 regulator categories identified in research, even if initial HS-range granularity is coarse — refine granularity in subsequent passes rather than blocking Phase 5 on perfect initial coverage.

---

## 13. Parallel Track B — CEPA Rules-of-Origin Data Collection

**What it is:** Encoding origin-qualification rules for the curated CEPA agreement subset that Phase 6 depends on.

**Suggested approach:**

1. Prioritize agreements by UAE trade volume relevance to likely early customers (India and Indonesia are reasonable starting candidates per earlier research, but this should be validated against actual pilot customer trade lanes once Phase 8 begins — per the PRD's open question on this)
2. For each agreement, encode: the general rule-of-origin test (regional value content threshold vs. change-in-tariff-classification), any product-specific rules that override the general rule for specific HS headings, and required supporting documentation
3. Same versioned-data-file approach and recurring-review discipline as Track A

**Target for Phase 6 readiness:** At least 2–3 agreements with reasonably complete rules before Phase 6 engineering begins, to give the `CEPAOriginService` real data to be built and tested against.

---

## 14. Sequencing Rationale (Why This Order)

- **Classification before consistency/permits/CEPA:** it's the hardest technical risk and



the core differentiator — validating it early means finding out if the core bet works before investing in everything built around it

- **Consistency before permit triage:** consistency-checking is pure computation with no data-research dependency, so it's a "free" phase to slot in while Track A is still maturing
- **Permit triage before CEPA:** permit triage's rules are comparatively simpler (category → regulator lookup) than CEPA's rules-of-origin logic (value-content math, change-in-classification tests) — building the simpler rules-engine pattern first makes the second, harder one faster to build
- **Both data tracks start immediately, not when their corresponding engineering phase begins:** research/data-entry work has a long lead time and doesn't require the founder's engineering hours, so running it in parallel from day one is the single biggest schedule risk mitigation available in this plan

---

## 15. What This Plan Deliberately Does Not Include

Per the architecture doc's Section 42 (avoiding overengineering) and Section 6 (non-goals in the PRD), this plan does not include phases for: direct Mirsal 2 filing integration, freight booking/tracking features, multi-country expansion, or splitting any module into an independently-deployed service. These are explicitly future extensions (architecture doc Section 32), not part of this build sequence — introducing them earlier would extend the solo founder's time-to-first-pilot-customer without validating the core product bet first.

---

## 16. Relationship to Other Project Documents

|                                                  |                               |
|--------------------------------------------------|-------------------------------|
| docs/                                            |                               |
| ├─ PRD.md                                        | ← what & why                  |
| ├─ SOFTWARE_ARCHITECTURE_AND_TECHNICAL_DESIGN.md | ← how it's structured         |
| ├─ API_SPEC.md                                   | ← endpoint contracts          |
| └─ IMPLEMENTATION_PLAN.md                        | ← this document — the build o |

This plan assumes the architecture and API contracts in the other two documents as fixed inputs; if either changes materially during a phase (e.g., a data-model change discovered while building Phase 3), update the source document first, then adjust this plan's remaining phases accordingly, rather than letting the two drift out of sync.